Django Class-Based Views (CBV) — A to Z Complete Notes

Topics: Django Views, Generic Views, CRUD, Mixins, Authentication, Method Overriding, Request Lifecycle, Query Optimization

Level: Beginner to Advanced

Language: English

---

1. Introduction to Class-Based Views (CBV)

1.1 What Is a Class-Based View?

A **Class-Based View (CBV)** is a Django view implemented using a Python class to handle HTTP requests and return HTTP responses.

Django supports two main approaches to writing views:

1. Function-Based Views (FBV)
2. Class-Based Views (CBV)

Function-Based View (FBV)

```
from django.http import HttpResponse


def home(request):
    return HttpResponse("Hello, Django!")
```

Class-Based View (CBV)

```
from django.http import HttpResponse
from django.views import View


class HomeView(View):
    def get(self, request):
        return HttpResponse("Hello, Django!")
```

In this example, `HomeView` is a class, and its `get()` method handles GET requests.

1.2 Advantages of CBV

- **Code Reusability:** Reuse functionality across multiple views.

- **Inheritance:** Extend existing view classes.
- **Mixins:** Add reusable behaviors such as authentication and permissions.
- **Generic Views:** Implement common functionality with less code.
- **Maintainability:** Organize complex applications more effectively.
- **Method Overriding:** Customize the default behavior of Django views.
- **CRUD Support:** Simplify create, read, update, and delete operations.

1.3 FBV vs. CBV

Feature	FBV	CBV
Structure	Python function	Python class
Request handling	Conditional statements	HTTP handler methods
Code reuse	Functions and helpers	Inheritance and mixins
Generic views	Must implement common behavior manually	Built-in generic views
Simple endpoints	Often straightforward	May require more structure
Complex CRUD	More manual	Often less code

	code	

Note: Neither approach is always better. Choose the approach that best fits the requirements.

---

2. Basic Class-Based View

Django's `View` class is the base class for standard CBVs.

```
from django.http import HttpResponse
from django.views import View


class MyView(View):

    def get(self, request, *args, **kwargs):
        return HttpResponse("This is a GET request.")

    def post(self, request, *args, **kwargs):
        return HttpResponse("This is a POST request.")
```

2.1 URL Configuration

```
from django.urls import path
from .views import MyView

urlpatterns = [
    path("myview/", MyView.as_view(), name="myview"),
]
```

2.2 Common HTTP Handler Methods

Method	Purpose
`get()`	Handles GET requests

`post()`	Handles POST requests
`put()`	Handles PUT requests
`patch()`	Handles PATCH requests
`delete()`	Handles DELETE requests
`head()`	Handles HEAD requests
`options()`	Handles OPTIONS requests
`trace()`	Handles TRACE requests if explicitly supported

These methods can be overridden when implementing a custom view. Django does not automatically provide application-specific implementations for every HTTP method.

---

3. Understanding `as_view()`

The `as_view()` method is essential when connecting a CBV to a URL pattern.

Example:

```
path("home/", HomeView.as_view(), name="home")
```

`as_view()` returns a callable that Django's URL resolver can invoke when a request matches the URL.

3.1 CBV Request Lifecycle

```
Browser Request
       |
       v
URL Resolver
       |
       v
HomeView.as_view()
       |
       v
Create View Instance
       |
       v
setup(request, ...)
       |
       v
dispatch(request, ...)
       |
       v
get() / post() / put() / delete()
       |
       v
HTTP Response
       |
       v
Browser
```

3.2 Important Base View Methods

`as_view()`

Creates a callable view from the class.

`setup()`

Initializes the view instance with the current request and URL arguments.

`dispatch()`

Selects the appropriate handler based on the HTTP request method.

`http_method_not_allowed()`

Returns an HTTP 405 Method Not Allowed response when the request method is not supported.

`options()`

Handles OPTIONS requests. Django's base view provides default handling for this method.

3.3 Overriding `dispatch()`

`dispatch()` can be overridden to execute shared logic before the request reaches its HTTP handler.

```
from django.http import HttpResponse
from django.views import View


class MyView(View):

    def dispatch(self, request, *args, **kwargs):
        # Common preprocessing logic
        return super().dispatch(request, *args, **kwargs)

    def get(self, request, *args, **kwargs):
        return HttpResponse("GET request")
```

Use `super().dispatch()` when you want Django's normal dispatch behavior to continue.

---

4. Generic Class-Based Views

Django provides built-in generic views for common web application tasks.

The main generic views are:

- `TemplateView`
- `RedirectView`
- `FormView`
- `ListView`

- `DetailView`
- `CreateView`
- `UpdateView`
- `DeleteView`

They reduce repetitive code and make applications easier to maintain.

4.1 TemplateView

`TemplateView` renders a template, optionally with additional context data.

```
from django.views.generic import TemplateView


class HomeView(TemplateView):
    template_name = "home.html"
```

URL Configuration

```
path("", HomeView.as_view(), name="home")
```

Template: `home.html`

```
<!DOCTYPE html>
<html>
<head>
    <title>Home</title>
</head>
<body>
    <h1>Welcome to Django</h1>
</body>
</html>
```

Adding Context Data

```
class HomeView(TemplateView):
    template_name = "home.html"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["message"] = "Welcome to Django CBV"
        return context
```

Template:

```
<h1>{{ message }}</h1>
```

Use cases: Home pages, About pages, Contact pages, and other template-oriented pages.

4.2 RedirectView

`RedirectView` redirects a user to another URL.

Using a URL

```
from django.views.generic import RedirectView


class HomeRedirectView(RedirectView):
    url = "/home/"
```

Using a URL Pattern Name

```
class HomeRedirectView(RedirectView):
    pattern_name = "home"
```

URL Configuration

```
path(
    "go-home/",
    HomeRedirectView.as_view(),
    name="go-home",
)
```

`url` specifies a destination URL, while `pattern_name` refers to a named URL pattern.

4.3 FormView

`FormView` handles form display, validation, and successful form submission. It does not automatically save a Django model instance.

```
from django.urls import reverse_lazy
from django.views.generic import FormView
from .forms import ContactForm


class ContactView(FormView):
```

```
        template_name = "contact.html"
        form_class = ContactForm
        success_url = reverse_lazy("contact-success")

        def form_valid(self, form):
            # Add custom processing here.
            return super().form_valid(form)
```

Important Attributes and Methods

Name	Purpose
`template_name`	Specifies the template
`form_class`	Specifies the form class
`success_url`	Defines the redirect destination
`form_valid()`	Handles valid form submissions
`form_invalid()`	Handles invalid form submissions

`get_form()`	Creates a form instance
`get_initial()`	Provides initial form values

Use `CreateView` or `UpdateView` when you want Django to handle saving a model instance through a ModelForm.

---

5. CRUD Operations with Generic Views

CRUD stands for:

- **Create:** Create a new object.
- **Read:** Retrieve and display objects.
- **Update:** Modify an existing object.
- **Delete:** Remove an existing object.

Django provides four main model-based CRUD views, along with `ListView` and `DetailView` for reading data.

The examples below assume that a `Product` model exists with fields such as `name`, `price`, and `active`.

5.1 ListView

`ListView` retrieves multiple objects and displays them in a template.

```
from django.views.generic import ListView
from .models import Product


class ProductListView(ListView):
    model = Product
    template_name = "products/product_list.html"
    context_object_name = "products"
```

URL Configuration

```
path(
    "products/",
    ProductListView.as_view(),
    name="product-list",
)
```

Template: `products/product_list.html`

```
<h1>Product List</h1>

{% for product in products %}
    <p>{{ product.name }} - {{ product.price }}</p>
{% empty %}
    <p>No products found.</p>
{% endfor %}
```

Important Attributes

Attribute	Purpose
`model`	Specifies the model
`queryset`	Specifies the queryset
`template_name`	Specifies the template
`context_object_name`	Sets the object-list variable name
`paginate_by`	Sets the number of

	objects per page

By default, the template receives `object_list`. Django can also provide a model-specific context name, such as `product_list`, when applicable.

Pagination Example

```
class ProductListView(ListView):
    model = Product
    template_name = "products/product_list.html"
    context_object_name = "products"
    paginate_by = 10
```

Template pagination can use `page_obj`, `paginator`, and the object list.

5.2 DetailView

`DetailView` retrieves and displays one object.

```
from django.views.generic import DetailView
from .models import Product


class ProductDetailView(DetailView):
    model = Product
    template_name = "products/product_detail.html"
    context_object_name = "product"
```

URL Configuration

```
path(
    "products/<int:pk>/",
    ProductDetailView.as_view(),
    name="product-detail",
)
```

Template: `products/product_detail.html`

```
<h1>{{ product.name }}</h1>
<p>Price: {{ product.price }}</p>
```

Django uses the URL's `pk` or configured `slug` to retrieve the object. If the object does not exist, Django normally returns a 404 response.

5.3 CreateView

`CreateView` displays a form and creates a new model instance after successful validation.

```
from django.urls import reverse_lazy
from django.views.generic import CreateView
from .models import Product


class ProductCreateView(CreateView):
    model = Product
    fields = ["name", "price"]
    template_name = "products/product_form.html"
    success_url = reverse_lazy("product-list")
```

URL Configuration

```
path(
    "products/create/",
    ProductCreateView.as_view(),
    name="product-create",
)
```

CreateView Workflow

```
GET Request
     |
     v
Display Form
     |
     v
User Submits Form
     |
     v
POST Request
     |
     v
Form Validation
```

```
          |
          +---- Invalid ----> Display Errors
          |
          +---- Valid ------> Save Object
                                  |
                                  v
                               Redirect
```

`CreateView` generally uses a ModelForm generated from the model and the specified fields.

Security note: Explicitly define editable fields. Do not expose sensitive fields to user input.

5.4 UpdateView

`UpdateView` retrieves an existing object, displays its current values in a form, and updates it after valid submission.

```
from django.views.generic import UpdateView


class ProductUpdateView(UpdateView):
    model = Product
    fields = ["name", "price"]
    template_name = "products/product_form.html"
    success_url = reverse_lazy("product-list")
```

URL Configuration

```
path(
    "products/<int:pk>/update/",
    ProductUpdateView.as_view(),
    name="product-update",
)
```

`UpdateView` uses the URL parameter to identify the existing object.

5.5 DeleteView

`DeleteView` displays a confirmation page and deletes the object when the user submits the confirmation form.

```
from django.views.generic import DeleteView


class ProductDeleteView(DeleteView):
    model = Product
    template_name = "products/product_confirm_delete.html"
    success_url = reverse_lazy("product-list")
```

URL Configuration

```
path(
    "products/<int:pk>/delete/",
    ProductDeleteView.as_view(),
    name="product-delete",
)
```

Template: `products/product_confirm_delete.html`

```
<h1>Delete Product</h1>

<p>Are you sure you want to delete {{ object.name }}?</p>

<form method="post">
    {% csrf_token %}
    <button type="submit">Confirm Delete</button>
</form>
```

Important: `DeleteView` normally displays the confirmation page on GET and performs deletion on POST. Do not perform destructive operations through GET requests.

---

6. Date-Based Generic Views

Date-based generic views display objects according to their dates. They are useful for blogs, news websites, event listings, and archives.

These views are available in `django.views.generic.dates`.

6.1 ArchiveIndexView

Displays an archive index of date-based objects.

```
from django.views.generic.dates import ArchiveIndexView
from .models import Post
```

```
class PostArchiveView(ArchiveIndexView):
    model = Post
    date_field = "published_at"
```

6.2 YearArchiveView

Displays objects from a specified year.

```
from django.views.generic.dates import YearArchiveView


class PostYearArchiveView(YearArchiveView):
    model = Post
    date_field = "published_at"
    make_object_list = True
```

6.3 MonthArchiveView

Displays objects from a specified month.

```
from django.views.generic.dates import MonthArchiveView


class PostMonthArchiveView(MonthArchiveView):
    model = Post
    date_field = "published_at"
```

6.4 WeekArchiveView

Displays objects from a specified week.

```
from django.views.generic.dates import WeekArchiveView


class PostWeekArchiveView(WeekArchiveView):
    model = Post
    date_field = "published_at"
```

6.5 DayArchiveView

Displays objects from a specified day.

```
from django.views.generic.dates import DayArchiveView


class PostDayArchiveView(DayArchiveView):
    model = Post
    date_field = "published_at"
```

6.6 TodayArchiveView

Displays objects from the current day.

```
from django.views.generic.dates import TodayArchiveView


class TodayPostView(TodayArchiveView):
    model = Post
    date_field = "published_at"
```

Important Notes

- The `Post` model must have the specified `published_at` field.
- URL patterns must supply the appropriate date parameters for the selected archive view.
- Templates and date formats must match the application's URL configuration.
- These views are useful for date-oriented archives but are not required for every project.

---

7. Authentication Generic Views

Django provides built-in authentication views in:

```
django.contrib.auth.views
```

They support login, logout, password changes, and password reset workflows.

7.1 LoginView

```
from django.contrib.auth.views import LoginView


class UserLoginView(LoginView):
    template_name = "accounts/login.html"
```

The template must render the login form and include a CSRF token in the POST form.

7.2 LogoutView

```
from django.contrib.auth.views import LogoutView


class UserLogoutView(LogoutView):
    next_page = "/"
```

In modern Django versions, the built-in `LogoutView` logs users out through POST requests. Use a POST form for logout rather than a state-changing link.

7.3 PasswordChangeView

Allows an authenticated user to change their password.

```
from django.contrib.auth.views import PasswordChangeView
from django.urls import reverse_lazy


class MyPasswordChangeView(PasswordChangeView):
    template_name = "accounts/password_change.html"
    success_url = reverse_lazy("password-change-done")
```

7.4 PasswordChangeDoneView

Displays a confirmation page after a successful password change.

7.5 PasswordResetView

Starts the password-reset process and sends a reset email when configured correctly.

7.6 PasswordResetDoneView

Displays a confirmation page after the password-reset request is submitted.

7.7 PasswordResetConfirmView

Allows the user to set a new password through a valid reset link.

7.8 PasswordResetCompleteView

Displays the password-reset completion page.

Authentication Views Summary

View	Purpose
`LoginView`	User login
`LogoutView`	User logout
`PasswordChangeView`	Change password
`PasswordChangeDoneView`	Password change confirmation
`PasswordResetView`	Start password reset
`PasswordResetDoneView`	Reset request confirmation
`PasswordResetConfirmView`	Set a new password
`PasswordResetCompleteView`	Reset completion

You must configure the appropriate URL patterns, templates, email settings, and redirects for your application.

---

8. Mixins in Django CBV

8.1 What Is a Mixin?

A **mixin** is a class that adds reusable functionality to another class through inheritance.

Mixins are frequently used in Django CBVs for authentication, permissions, form handling, and object management.

8.2 LoginRequiredMixin

Restricts access to authenticated users.

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import ListView
from .models import Product


class DashboardView(LoginRequiredMixin, ListView):
    model = Product
    template_name = "dashboard.html"
    login_url = "/accounts/login/"
```

Unauthenticated users are normally redirected to the login page.

8.3 PermissionRequiredMixin

Checks whether the user has the required Django permission.

```
from django.contrib.auth.mixins import (
    LoginRequiredMixin,
    PermissionRequiredMixin,
)
from django.views.generic import DeleteView


class ProductDeleteView(
    LoginRequiredMixin,
    PermissionRequiredMixin,
    DeleteView,
):
    model = Product
```

```
    permission_required = "shop.delete_product"
    success_url = reverse_lazy("product-list")
```

Permission strings follow this format:

```
app_label.permission_codename
```

For example:

```
shop.delete_product
```

The permission must exist, and the application must use Django's permission system correctly.

8.4 UserPassesTestMixin

Checks a custom condition before allowing access.

```
from django.contrib.auth.mixins import (
    LoginRequiredMixin,
    UserPassesTestMixin,
)
from django.views.generic import UpdateView


class ProductUpdateView(
    LoginRequiredMixin,
    UserPassesTestMixin,
    UpdateView,
):
    model = Product
    fields = ["name", "price"]

    def test_func(self):
        product = self.get_object()
        return product.created_by == self.request.user
```

This example assumes the `Product` model contains a `created_by` field. It allows users to update only their own products.

8.5 Other Useful Mixins

Mixin	Purpose
`FormMixin`	Reusable form-handling behavior
`ModelFormMixin`	Form handling for model instances
`MultipleObjectMixin`	Multiple-object handling
`SingleObjectMixin`	Single-object handling
`ContextMixin`	Adds context-related behavior

Best practice: Put authentication mixins before the main generic view in the inheritance list.

---

9. Method Overriding in CBV

Method overriding means providing a custom implementation of a method inherited from a parent class.

It allows you to customize Django's default behavior.

9.1 Overriding `get_queryset()`

```
class ProductListView(ListView):
    model = Product

    def get_queryset(self):
        return Product.objects.filter(active=True)
```

Only active products are returned.

When extending the parent's queryset behavior, use `super()` when appropriate:

```
def get_queryset(self):
    queryset = super().get_queryset()
    return queryset.filter(active=True)
```

9.2 Overriding `get_context_data()`

```
class ProductListView(ListView):
    model = Product

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["message"] = "Welcome to the Product List"
        return context
```

Template:

```
<h1>{{ message }}</h1>
```

9.3 Overriding `form_valid()`

```
from django.urls import reverse_lazy
from django.views.generic import CreateView


class ProductCreateView(LoginRequiredMixin, CreateView):
    model = Product
    fields = ["name", "price"]
    success_url = reverse_lazy("product-list")

    def form_valid(self, form):
        form.instance.created_by = self.request.user
        return super().form_valid(form)
```

This assumes the `Product` model has a `created_by` field. The view also requires authentication because it uses `LoginRequiredMixin`.

9.4 Overriding `form_invalid()`

```
def form_invalid(self, form):
    # Add custom error handling if necessary.
    return super().form_invalid(form)
```

The default implementation normally re-renders the form with validation errors.

9.5 Overriding `get_object()`

Use `get_object()` to customize how a single object is retrieved.

```
class ProductDetailView(DetailView):
    model = Product

    def get_queryset(self):
        return Product.objects.filter(active=True)
```

Filtering `get_queryset()` is often preferable to manually implementing `get_object()`, because Django's existing object retrieval behavior remains available.

---

10. Important Form Handling Methods

Method	Purpose
`get_form_class()`	Determines the form class
`get_form()`	Creates the form instance
`get_form_kwargs()`	Provides arguments to the form

	constructor
`get_initial()`	Provides initial form values
`form_valid()`	Handles valid submissions
`form_invalid()`	Handles invalid submissions
`get_success_url()`	Determines the redirect destination

Example: Passing the Current User to a Form

In the view:

```
def get_form_kwargs(self):
    kwargs = super().get_form_kwargs()
    kwargs["user"] = self.request.user
    return kwargs
```

In the custom form:

```
from django import forms


class ProductForm(forms.ModelForm):
```

```
        def __init__(self, *args, user=None, **kwargs):
            super().__init__(*args, **kwargs)
            self.user = user
```

This example demonstrates passing extra initialization data. A complete `ModelForm` must also define its model and editable fields.

---

11. Decorators in CBV

Function-based decorators can be applied to CBV methods using Django's `method_decorator`.

11.1 Applying a Decorator to `dispatch()`

```
from django.contrib.auth.decorators import login_required
from django.utils.decorators import method_decorator
from django.http import HttpResponse
from django.views import View


@method_decorator(login_required, name="dispatch")
class MyView(View):

    def get(self, request, *args, **kwargs):
        return HttpResponse("Authenticated users only.")
```

Applying `login_required` to `dispatch()` protects the view's supported HTTP methods.

11.2 Applying a Decorator to a Specific Method

```
from django.utils.decorators import method_decorator
from django.contrib.auth.decorators import login_required
from django.views import View
from django.http import HttpResponse


class MyView(View):

    @method_decorator(login_required)
```

```
    def get(self, request, *args, **kwargs):
        return HttpResponse("Protected GET request.")
```

In this example, only the `get()` method has the decorator. Other HTTP handlers would require their own protection if necessary.

Recommendation: For ordinary CBV authentication, `LoginRequiredMixin` is often the clearer option.

---

12. Request Objects and URL Arguments

12.1 `self.request`

`self.request` contains the current HTTP request.

```
class ProductListView(ListView):
    model = Product

    def get_queryset(self):
        user = self.request.user
        return Product.objects.filter(created_by=user)
```

This example retrieves products created by the current user.

12.2 `self.kwargs`

`self.kwargs` contains named URL parameters.

URL configuration:

```
path(
    "products/category/<int:category_id>/",
    ProductListView.as_view(),
    name="products-by-category",
)
```

View:

```
class ProductListView(ListView):
    model = Product

    def get_queryset(self):
        category_id = self.kwargs["category_id"]
        return Product.objects.filter(category_id=category_id)
```

This assumes `Product` has a `category` ForeignKey.

12.3 `self.args`

`self.args` contains positional URL arguments. Named URL parameters accessed through `self.kwargs` are generally easier to understand and maintain.

---

13. Understanding `reverse_lazy()`

`reverse_lazy()` resolves a named URL lazily.

```
from django.urls import reverse_lazy



class ProductCreateView(CreateView):
    model = Product
    fields = ["name", "price"]
    success_url = reverse_lazy("product-list")
```

It is useful for class attributes because URL resolution can be deferred until the URL configuration is available.

Compare:

- `reverse()` resolves a URL immediately.
- `reverse_lazy()` defers URL resolution.

Use named URLs instead of hardcoding paths when practical.

---

14. Complete CRUD Example with Authentication and Authorization

This example demonstrates a Product CRUD application.

Assumptions:

- `Product` has `name`, `price`, `description`, `active`, and `created_by` fields.
- `created_by` is a ForeignKey to the user model.
- The appropriate migrations have been applied.
- The required templates exist.

14.1 ListView

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import ListView
from .models import Product


class ProductListView(LoginRequiredMixin, ListView):
    model = Product
    template_name = "products/product_list.html"
    context_object_name = "products"

    def get_queryset(self):
        return Product.objects.filter(active=True)
```

14.2 DetailView

```
from django.views.generic import DetailView


class ProductDetailView(LoginRequiredMixin, DetailView):
    model = Product
    template_name = "products/product_detail.html"
    context_object_name = "product"
```

This version restricts product detail access to authenticated users. Add further object-level restrictions if the application requires them.

14.3 CreateView

```
from django.urls import reverse_lazy
from django.views.generic import CreateView


class ProductCreateView(LoginRequiredMixin, CreateView):
    model = Product
    fields = ["name", "price", "description"]
    template_name = "products/product_form.html"
    success_url = reverse_lazy("product-list")

    def form_valid(self, form):
        form.instance.created_by = self.request.user
        return super().form_valid(form)
```

14.4 UpdateView

```
from django.contrib.auth.mixins import UserPassesTestMixin
from django.views.generic import UpdateView


class ProductUpdateView(
    LoginRequiredMixin,
    UserPassesTestMixin,
    UpdateView,
):
    model = Product
    fields = ["name", "price", "description"]
    template_name = "products/product_form.html"
    success_url = reverse_lazy("product-list")

    def test_func(self):
        return self.get_object().created_by == self.request.user
```

14.5 DeleteView

```
from django.views.generic import DeleteView


class ProductDeleteView(
    LoginRequiredMixin,
    UserPassesTestMixin,
    DeleteView,
):
    model = Product
    template_name = "products/product_confirm_delete.html"
    success_url = reverse_lazy("product-list")

    def test_func(self):
        return self.get_object().created_by == self.request.user
```

14.6 URL Configuration

```
from django.urls import path
from .views import (
    ProductListView,
```

```
        ProductDetailView,
        ProductCreateView,
        ProductUpdateView,
        ProductDeleteView,
    )

    urlpatterns = [
        path(
            "products/",
            ProductListView.as_view(),
            name="product-list",
        ),
        path(
            "products/create/",
            ProductCreateView.as_view(),
            name="product-create",
        ),
        path(
            "products/<int:pk>/",
            ProductDetailView.as_view(),
            name="product-detail",
        ),
        path(
            "products/<int:pk>/update/",
            ProductUpdateView.as_view(),
            name="product-update",
        ),
        path(
            "products/<int:pk>/delete/",
            ProductDeleteView.as_view(),
            name="product-delete",
        ),
    ]
```

Security Checklist

1. Ensure `created_by` exists in the model.
2. Apply authentication to views that require a logged-in user.
3. Check object ownership before updating or deleting records.
4. Define explicit editable fields.
5. Use POST forms and CSRF protection for state-changing actions.

6. Decide whether product details should be public or private.
7. Use Django's permission system where role-based access is needed.

---

15. Database Query Optimization

Django CBVs do not automatically optimize every database query. Query optimization must be applied according to the data relationships and actual query patterns.

15.1 `select_related()`

Use `select_related()` for ForeignKey and OneToOneField relationships.

```
class ProductListView(ListView):
    model = Product

    def get_queryset(self):
        return Product.objects.select_related("category")
```

This assumes `Product` has a ForeignKey named `category`.

15.2 `prefetch_related()`

Use `prefetch_related()` for ManyToManyField and reverse relationships.

```
class ProductListView(ListView):
    model = Product

    def get_queryset(self):
        return Product.objects.prefetch_related("tags")
```

This assumes `Product` has a relationship named `tags`.

15.3 Other Optimization Techniques

Technique	Purpose
`select_related()`	Joins single-valued relationships

`prefetch_related()`	Fetches related collections efficiently
`only()`	Loads selected fields initially
`defer()`	Defers loading selected fields
`paginate_by`	Limits objects displayed per page
Caching	Reuses eligible previously computed results

Measure actual queries before optimizing. Django Debug Toolbar can help identify unnecessary database queries during development.

---

16. Returning JSON Responses from CBV

CBVs can return JSON instead of rendering HTML.

```
from django.http import JsonResponse
from django.views import View
```

```
class ProductAPIView(View):

    def get(self, request, *args, **kwargs):
        return JsonResponse({
            "message": "Success",
            "status": "ok",
        })
```

`JsonResponse` serializes a Python dictionary into a JSON HTTP response.

For more complex REST APIs, consider Django REST Framework, which provides API-specific views, serializers, authentication, permissions, and other API features.

---

17. Important CBV Methods — Quick Reference

Base View

Method	Purpose
`as_view()`	Creates a callable view
`setup()`	Initializes request attributes
`dispatch()`	Routes the request to a handler
`http_method_not_allowed()`	Handles unsupported methods

`options()`	Handles OPTIONS requests

ListView

Method	Purpose
`get_queryset()`	Returns the object queryset
`get_context_data()`	Adds template context
`get_paginate_by()`	Determines pagination size
`paginate_queryset()`	Applies pagination
`render_to_response()`	Creates a template response

DetailView

Method	Purpose

`get_object()`	Retrieves one object
`get_queryset()`	Defines eligible objects
`get_context_data()`	Adds template context

CreateView and UpdateView

Method	Purpose
`get_form_class()`	Determines the form class
`get_form()`	Creates the form instance
`get_form_kwargs()`	Supplies form constructor arguments
`get_initial()`	Provides initial form data

`form_valid()`	Handles valid form submissions
`form_invalid()`	Handles invalid form submissions
`get_success_url()`	Determines the success redirect

DeleteView

Method	Purpose
`get_object()`	Retrieves the object
`get_queryset()`	Defines eligible objects
`get_context_data()`	Adds template context

`get_success_url()`	Determines the success redirect
`post()`	Processes the confirmation submission

---

18. Common Mistakes in Django CBV

Mistake 1: Forgetting `as_view()`

Incorrect:

```
path("products/", ProductListView)
```

Correct:

```
path("products/", ProductListView.as_view())
```

Mistake 2: Assuming FormView Automatically Saves a Model

`FormView` handles form processing but does not automatically create or update a model instance.

Use `CreateView`, `UpdateView`, or explicit save logic when necessary.

Mistake 3: Forgetting to Define Editable Fields

Incorrectly configured `CreateView` and `UpdateView` classes may fail to construct their ModelForms.

Define `fields` or a suitable `form_class`.

Mistake 4: Replacing Parent Context Accidentally

Avoid:

```
    def get_context_data(self, **kwargs):
        return {"message": "Hello"}
```

This discards the existing context.

Prefer:

```
    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["message"] = "Hello"
        return context
```

Mistake 5: Forgetting `super()`

When overriding a method, skipping the parent implementation may remove important behavior.

Use `super()` when the inherited behavior should remain part of the execution.

Mistake 6: Confusing Authentication with Authorization

Authentication answers: **Who is the user?**

Authorization answers: **What is the user allowed to do?**

A logged-in user should not automatically be allowed to modify every record.

Mistake 7: Performing Deletion Through GET

Use a POST confirmation form with CSRF protection for destructive operations.

Mistake 8: Assuming Query Optimization Is Automatic

Use `select_related()` or `prefetch_related()` where appropriate, and inspect the actual queries.

---

19. Interview and Exam Questions

Basic Questions

1. What is a Class-Based View in Django?
2. What is the difference between FBV and CBV?
3. What is the purpose of `as_view()`?
4. What does `dispatch()` do?
5. What is a generic view?
6. Why is `super()` useful in CBVs?

Generic Views

7. What is the difference between `ListView` and `DetailView`?
8. What is the difference between `CreateView` and `UpdateView`?
9. How does `DeleteView` work?
10. When should you use `TemplateView`?
11. What is the purpose of `FormView`?
12. What is `get_queryset()`?
13. What is `get_object()`?
14. How does `get_context_data()` work?
15. What is the purpose of `form_valid()`?

Mixins and Security

16. What is a mixin?
17. What is the difference between `LoginRequiredMixin` and `PermissionRequiredMixin`?
18. When should you use `UserPassesTestMixin`?
19. Why are authentication and authorization different?
20. How can you restrict users to updating their own objects?
21. Why should state-changing operations not use GET requests?

Advanced Questions

22. Explain the CBV request lifecycle.
23. How can you pass extra data to a form?
24. What is the difference between `reverse()` and `reverse_lazy()`?
25. How do `select_related()` and `prefetch_related()` differ?
26. How do you return JSON from a CBV?
27. When should you choose FBV instead of CBV?
28. How can you customize a generic view's queryset?
29. How does pagination work in `ListView`?
30. How can decorators be applied to CBVs?

---

20. Final Revision Sheet

Core Views

View	Purpose
`View`	Basic request handling
`TemplateView`	Render a template
`RedirectView`	Redirect to another URL
`FormView`	Process forms
`ListView`	Display multiple objects
`DetailView`	Display one object
`CreateView`	Create an object
`UpdateView`	Update an object
`DeleteView`	Delete an object

Date-Based Views

- `ArchiveIndexView` — Archive index
- `YearArchiveView` — Yearly archive
- `MonthArchiveView` — Monthly archive

- `WeekArchiveView` — Weekly archive
- `DayArchiveView` — Daily archive
- `TodayArchiveView` — Today's archive

Authentication Views

- `LoginView` — User login
- `LogoutView` — User logout
- `PasswordChangeView` — Change password
- `PasswordChangeDoneView` — Password change confirmation
- `PasswordResetView` — Start password reset
- `PasswordResetDoneView` — Reset request confirmation
- `PasswordResetConfirmView` — Set a new password
- `PasswordResetCompleteView` — Reset completion

Important Mixins

- `LoginRequiredMixin` — Require authentication
- `PermissionRequiredMixin` — Check Django permissions
- `UserPassesTestMixin` — Check a custom condition
- `FormMixin` — Reusable form handling
- `ModelFormMixin` — Model-form behavior
- `MultipleObjectMixin` — Multiple-object handling
- `SingleObjectMixin` — Single-object handling
- `ContextMixin` — Context-related behavior

Important Methods

- `as_view()` — Creates a callable view
- `setup()` — Initializes the view
- `dispatch()` — Routes HTTP requests
- `get_queryset()` — Customizes the queryset
- `get_object()` — Retrieves one object
- `get_context_data()` — Adds template context
- `get_form()` — Creates a form instance
- `form_valid()` — Handles valid form submissions
- `form_invalid()` — Handles invalid form submissions
- `get_success_url()` — Determines the success redirect

CRUD Formula

```
CREATE  -> CreateView
READ    -> ListView / DetailView
UPDATE  -> UpdateView
DELETE  -> DeleteView
```

CBV Request Flow

```
URL Pattern
    |
    v
as_view()
    |
    v
View Instance
    |
    v
setup()
    |
    v
dispatch()
    |
    v
HTTP Handler
(get / post / put / patch / delete)
    |
    v
Generic View Logic
    |
    v
HTTP Response
```

Final Summary

Django Class-Based Views combine Python classes, inheritance, mixins, and generic views to make web application development more reusable and maintainable.

To master CBVs, learn these concepts in order:

1. `View`, `as_view()`, `setup()`, and `dispatch()`
2. `TemplateView`, `RedirectView`, and `FormView`

3. `ListView`, `DetailView`, `CreateView`, `UpdateView`, and `DeleteView`
4. Method overriding and context management
5. Authentication, permissions, and object-level authorization
6. Query optimization, pagination, and JSON responses

The key principle: Understand the default behavior of a generic view first, then override only the methods needed to customize it.